

Global or Local Adaptation? Client-Sampled Federated Meta-Learning for Personalized IoT Intrusion Detection

Haorui Yan, Xi Lin[✉], *Member, IEEE*, Shenghong Li[✉], *Senior Member, IEEE*, Hao Peng[✉], and Bo Zhang

Abstract—With the increasing size of Internet of Things (IoT) devices, cyber threats to IoT systems have increased. Federated learning (FL) has been implemented in an anomaly-based intrusion detection system (NIDS) to detect malicious traffic in IoT devices and counter the threat. However, current FL-based NIDS mainly focuses on global model performance and lacks personalized performance improvement for local data. To address this issue, we propose a novel personalized federated meta-learning intrusion detection approach (PerFLID), which allows multiple participants to personalize their local detection models for local adaptation. PerFLID shifts the goal of the personalized detection task to training a local model suitable for the client's specific data, rather than a global model. To meet the real-time requirements of NIDS, PerFLID further refines the client selection strategy by clustering the local gradient similarities to find the nodes that contribute the most to the global model per global round. PerFLID can select the nodes that accelerate the convergence of the model, and we theoretically analyze the improvement in the convergence speed of this strategy over the personalized federated learning algorithm. We experimentally evaluate six existing FL-NIDS approaches on three real network traffic datasets and show that our PerFLID approach outperforms all baselines in detecting local adaptation accuracy by 10.11% over the state-of-the-art scheme, accelerating the convergence speed under various parameter combinations.

Index Terms—Internet of Things security, personalized traffic intrusion detection, federated meta-learning, client selection aggregation.

I. INTRODUCTION

THE Internet of Things (IoT) is a network system that connects physical devices and machines to the Internet

using information sensors and communication technologies. This enables IoT devices to communicate and exchange data, providing services for scenarios such as smart homes, industrial automation, smart healthcare, and smart transportation [1], [2], [3], [4].

However, serious security issues arise when IoT devices are compromised by malicious intruders and participate in data interaction and processing. Intruders can penetrate the network layer through which devices and external networks exchange data, and threaten IoT devices that lack security defenses. Since 2016, the well-known Mirai botnet has initiated numerous large-scale distributed denial-of-service (DDoS) attacks against IoT devices [5]. The BlueBorne vulnerability, identified in 2017, affects nearly all Bluetooth-enabled devices and poses a significant threat to IoT devices [6].

Due to the limited processing power and storage capacity of many IoT device systems, sophisticated security systems are frequently feasible. As a result, intrusion detection techniques have been employed to mitigate unknown attacks in the network [7]. However, traffic-based intrusion detection techniques have several limitations, including insufficient data in public traffic sets and difficulty in detecting IoT attacks such as DDoS, exploits, reconnaissance, and worms [8]. Therefore, relying solely on this approach to secure IoT devices is unreliable. Centralized detection systems are unsuitable for IoT architectures with a large number of distributed components, making it difficult to detect single points of failure in IoT. Additionally, detection techniques must minimize latency to meet the frequent data processing demands of IoT devices, while ensuring robust data privacy. As IoT traffic continues to increase, traditional intrusion detection systems face significant challenges in providing fast detection due to their high computational overhead [9], [10]. Recent studies review conventional detection methods and emphasize that rapid detection has become one of the primary challenges in the field of network intrusion detection [11].

According to current research, federated learning (FL) based intrusion detection systems (IDS) could provide a solution to some of these issues due to their improved adaptability and scalability [12]. FL schemes allow users to jointly train detection models without sharing their private data, addressing the problem of insufficient public data in IoT scenarios while protecting data privacy [13]. This contrasts with non-federated machine learning methods, which do not offer this level of privacy protection and may require a larger amount of data to be effective [14], [15].

Received 5 April 2024; revised 5 August 2024 and 18 November 2024; accepted 20 November 2024. Date of publication 12 December 2024; date of current version 26 December 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62202302 and Grant 62372297. The associate editor coordinating the review of this article and approving it for publication was Prof. Ghassan Karame. (*Corresponding author: Xi Lin.*)

Haorui Yan, Xi Lin, and Shenghong Li are with the the School of Cyber Security, Shanghai Jiao Tong University, Shanghai 200240, China, and also with Shanghai Key Laboratory of Integrated Administration Technologies for Information Security, Shanghai 200240, China (e-mail: yhr0322@sjtu.edu.cn; linxi234@sjtu.edu.cn; shli@sjtu.edu.cn).

Hao Peng is with the Key Laboratory of Intelligent Education Technology and Application of Zhejiang Province and the School of Computer Science and Technology, Zhejiang Normal University, Jinhua 321004, China (e-mail: hpeng@zjnu.edu.cn).

Bo Zhang is with the School of Cyber Science and Engineering, Shanghai Jiao Tong University, Shanghai 200240, China, and also with the State Grid Smart Grid Research Institute Company Ltd., State Grid Corporation of China, Nanjing 210000, China (e-mail: bozhangit@sjtu.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TIFS.2024.3516548>, provided by the authors.

Digital Object Identifier 10.1109/TIFS.2024.3516548

1556-6021 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

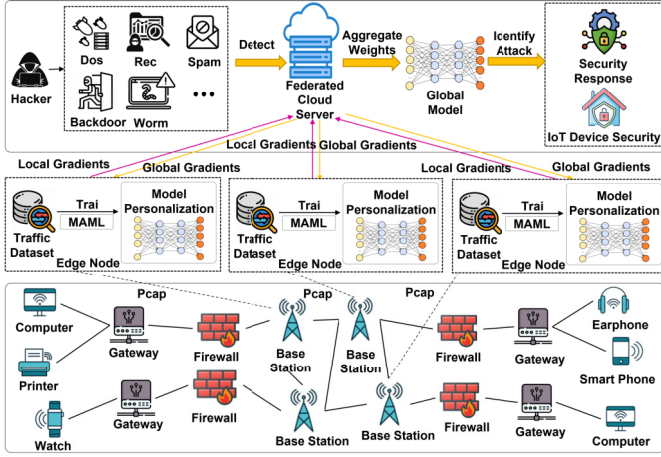


Fig. 1. A framework for traffic intrusion detection of IoT devices based on PFL.

Although FL is a detection scheme for detecting various types of cyber threats to the IoT, there are still some unresolved problems. Li et al. [16] proposed an FL framework to build an IoT intrusion detection model collaboratively. Chatterjee and Hanawal [17] proposed an FL-based intrusion detection system that uses a hybrid federated averaging (FedAvg) and noise-resistant integration framework to deal with labeling noise. Liu et al. [18] used blockchain technology for storing and sharing FL models, which is used to ensure the security of the aggregated detection models. These studies improved the structure of the detection model by utilizing existing FL algorithms and presented the application of FL to IDS in the context of the IoT. However, it is important to note that these studies assume a uniform distribution of client data, and the goal is to train an optimal global model. Models trained based on this assumption lack local data adaptation. First, the data is non-independently and identically distributed (Non-IID) among clients, which is due to the fact that IoT devices are produced by different manufacturers and configured with different protocols, resulting in varying traffic data from different devices in different network scenarios. As a result, the global model obtained from training cannot adapt to these data. Personalized Federated Learning (PFL) is an approach to address the problem of slow convergence and poor performance on Non-IID data as well as the lack of personalization of the model for local tasks or datasets [19]. One of the methods to achieve personalization is meta-learning, which optimizes the global model to enable fast convergence of user-local training by treating the training phase of FL as the training phase of meta-learning, and the personalization phase of the FL model as the testing phase of meta-learning [20].

In response to this, we propose a novel framework for intrusion detection in IoT traffic, termed Personalized Federated Learning Intrusion Detection (PerFLID). To simplify the description, we replace the client in FL with an IoT edge node. PerFLID enables the training of local models on edge nodes with different traffic data distributions in the IoT. In contrast to global models, the locally trained models exhibit enhanced personalization, making them optimal for intrusion detection.

Specifically, we sample and extract features from the traffic information of each edge node, treating intrusion detection as a feature classification task with each edge node's detection as a subtask. The central server sends malicious traffic detection models to the edge nodes, which are then trained collaboratively using a federated meta-learning approach. During the training process of each subtask, the edge nodes execute the model-agnostic meta-learning (MAML) algorithm locally to update parameters while retaining local feature information. With this approach, the global model converges with only a few rounds of gradient descent at the edge nodes, resulting in higher accuracy for personalized intrusion detection. To enable fast convergence during model training, we design a node selection strategy based on gradient feature clustering. The strategy clusters the edge nodes based on their gradient features in each communication round, calculates the probability of each node being selected based on the feature distribution, and samples the optimal subset of nodes participating in the aggregation. The representative gradient features of the edge nodes are preserved during the aggregation process at the central server, and each training round is run only once at the server, thus not introducing additional communication overhead. The main contributions of this paper are as follows:

- We propose a Personalized Federated Learning Intrusion Detection Framework, enabling personalized intrusion detection for IoT devices facing attacks with unknown traffic types. This method leverages meta-learning to develop personalized models, effectively adapting to local data characteristics.
- We design a node selection strategy that selects nodes based on the clustering results of local gradient features. This strategy is theoretically proven to accelerate the convergence speed during model training, thereby enhancing the overall efficiency of our framework.
- We demonstrate that our proposed personalized traffic intrusion detection framework has a convergence upper bound. Furthermore, we conduct extensive simulation experiments with three neural network models on three NIDS datasets to verify that the proposed framework achieves higher detection accuracy with local adaptation and faster training convergence.

The rest of this paper is organized as follows: In Section II, we review the related work of FL and traffic intrusion detection and summarize the PFL approach. In Section III, we illustrate the underlying architecture with implementation details. In Section IV, we introduce our PerFLID workflow. In Section V, We perform a theoretical convergence analysis of PerFLID. In Section VI, we present and discuss our evaluation results. Finally, we present the conclusion in Section VII.

II. RELATED WORK

A. Federated Learning Based Traffic Intrusion Detection

Nguyen et al. proposed DiOT [15], a self-learning distributed system for detecting IoT devices infected with Mirai malware. This intrusion detection system uses FL methods for the first time, enabling distributed learning of models across multiple clients. However, this method only targets

Mirai malware attacks and lacks a complete implementation of the intrusion detection FL framework. Later, several researchers applied different FL models to solve the IoT intrusion detection problem. Mothukuri et al. [12] proposed an anomaly detection method with FL, which protects user data privacy by training anomaly detection machine learning models on IoT devices without transmitting user data to a centralized server. They used long short-term memory (LSTM) and gated recurrent unit (GRU) neural network models to train ML models on Modbus network datasets. The results show the lowest error rate in predicting attacks and fewer false alarms than centralized machine learning. Aouedi et al. [21] implemented a semi-supervised learning scheme for intrusion detection in FL by training a local autoencoder (AE) to learn the features of intrusion data. Zhao et al. [22] proposed a network anomaly detection method based on FL and transfer learning to address the issue of scarce training data for network anomaly detection. The model achieved a success rate of 97.23% in detecting vulnerability attacks on the UNSW-NB15 dataset. To quickly detect APT attacks with privacy protection, Hu et al. [23] proposed a traffic detection method based on coalitional meta-learning. The method achieves a detection accuracy distribution of 86.67% and 67.6% on the CIC-IDS2017 and DAPT2020 datasets, respectively. Ding et al. [24] proposed a scalable NIDS for large-scale IoT networks. This system leverages a meta-learning framework to optimize the parallelism of GNN-based NIDS and introduces a coalition formation strategy to enhance the accuracy and reduce the communication overhead of the NIDS.

B. Personalized Federated Learning in IoT

Tan et al. [19] proposed to tackle two challenges in federated learning: slow convergence and subpar performance when dealing with highly heterogeneous (Non-IID) data, as well as the absence of personalized models for local tasks or datasets. Luca et al. [25] introduced the use of data augmentation to address the issue of out-of-domain generalization in federated learning. They demonstrated that suitable data augmentation could mitigate Non-IID effects in FL, thereby enhancing FL generalization and convergence from a causal perspective. Fraboni et al. [26] propose a jointly learned client selection strategy based on cluster selection. Compared to multinomial distribution (MD) sampling, this strategy is both unbiased and reduces the variance of random client aggregation. Implementing personalized federated learning includes a variety of model-based approaches in addition to data-based approaches, one of which is meta-learning. Meta-learning aims to enable models to rapidly learn new tasks based on existing knowledge. The MAML algorithm [27] possesses robust generalization capabilities and can be applied in various gradient descent methods, such as supervised learning and reinforcement learning, hence its designation as a model-independent meta-learning algorithm. Fallah et al. [28] proposed Per-FedAvg within FedAvg using MAML equations. They demonstrated that meta-learning can enhance the training of shared initial global models. Participants involved in the

training or new clients only need to execute a few steps of gradient descent on local data. This enables adaptation to the local dataset and ultimately yields a more personalized model.

In our paper, we address the lack of local data adaptation in existing federated learning intrusion detection work by proposing a locally personalized scheme based on federated meta-learning. We provide detailed theoretical proof to support our approach. Additionally, to expedite the deployment of federated meta-learning, we have optimized the client aggregation strategy. This optimization accelerates the deployment of meta-learning without incurring additional resource overhead, and we have also provided theoretical proof for this improvement.

III. PRELIMINARIES

A. Federated Meta Learning

Firstly, let us briefly review the formulation of MAML. The core idea of MAML is to learn an effective initial set of model parameters, enabling the model to adapt quickly and perform well on new tasks with only a few gradient updates. In the context of federated learning, for each participating user, if we assume each user takes the initial point and updates it using one step of gradient descent based on its own loss function, the federated learning global optimization objective $F(w)$ is defined as follows:

$$\min_{w \in \mathbb{R}^d} F(w) := \frac{1}{N} \sum_{i=1}^N f_i(w - \alpha \nabla f_i(w)). \quad (1)$$

Here, the local dataset D_i is generated by post-processing the traffic files sampled from IoT devices, which is then used to train the local model parameters w_i . The individual loss function is defined as $f_i(w)$, and the parameters w_i are updated via gradient descent with respect to f_i . The step size $\alpha \geq 0$ represents the learning rate utilized by the node for training the model.

The advantage of this computational method is that both existing and new users can adopt it by using the solution of the new problem as an initial point and slightly adjusting it based on their own data. This means that users can utilize the initialized result and then update it with their dataset D_i by performing only one or a few gradient descent steps. Users define their metafunctions locally as follows:

$$F_i(w) := f_i(w - \alpha \nabla f_i(w)). \quad (2)$$

Following the completion of global model aggregation by the server, the updated model is dispatched to the node to commence a fresh round of meta-learning training. Through multiple rounds of communication, the initial model $F(w)$ is attained. A detailed discussion on the model update algorithm will be provided in Section IV.

B. Data Heterogeneity

We define the traffic dataset $D = \{(x, y)\}$ as the global dataset. Here, x represents the features extracted from network traffic that includes benign traffic as well as the latest common network attacks (such as Fuzzer, Backdoor, Exploits, DDoS,

etc.), including attributes such as timestamps, source and destination IP addresses, source and destination ports, and protocol type. The label y corresponds to x , representing the type of network attack. Consider N IoT devices, labeled as $C_1, C_2, \dots, C_n$, acting as nodes. Each node has its own local dataset, denoted as $D_i = \{(x_i, y_i)\}$. Similarly, the label y_i of the local data from edge IoT devices corresponds to the type of network attack occurring within the local network of the device. Notate the local dataset size and global dataset size with $|D_i|$ and $|D|$ respectively. Here, $|D| = \sum_{i \in N} |D_i|$ denotes the total number of global samples equals the sum of local samples. The distribution of traffic data from real IoT devices is diverse, making it challenging to construct meaningful federated traffic datasets due to privacy concerns. Following the suggestion by Li et al. [29], dividing the real dataset into a distributed Non-IID dataset helps balance the local data across nodes while facilitating effective FL experiments. In this section, we discuss how to partition the traffic dataset. Initially, we assume that traffic data on any device is associated with one label. We define the local data distribution $P(x_i, y_i) = P(x_i|y_i)P(y_i)$ or $P(x_i, y_i) = P(y_i|x_i)P(x_i)$. Drawing from Non-IID classification [30], we propose three methods for partitioning federated traffic data:

1) *Label Imbalance*: In most IoT devices, the types of attacks originating from device vulnerabilities are typically limited, resulting in different label distributions for each node's dataset, i.e., the distribution $P(y_i)$ varies across nodes. To quantify this, we use the matrix $\mathbf{X} \in \mathbb{R}^{K \times N}$ as the label distribution matrix, where K represents the number of attack categories and N represents the number of nodes. Each row vector $\mathbf{x}_k \in \mathbb{R}^N$ represents the probability distribution of category k across different nodes, with each dimension of the vector indicating the proportion of samples from category k assigned to different nodes. To define a random variable for each category and allow for the adjustment of the distribution's balance, we use the Dirichlet distribution, which is suitable for multivariate random variables and has been widely discussed in studies of data heterogeneity in FL [31]. Therefore, we sample from $P_K \sim \text{DP}(\alpha, G_0)$ and select a random vector as the probability distribution vector $\mathbf{x}_k \in \mathbb{R}^N$ for the labels. Here, G_0 represents the base distribution for a single node, and the imbalance can be adjusted through the parameter α , where smaller values of α result in greater imbalance.

2) *Feature Imbalance*: Abnormal activities in IoT traffic data significantly differ from normal system activities, even within data sampled from the same device, abnormal and normal traffic data possess different traits. We denote these feature imbalances as $P(x_i)$ distribution imbalances.

3) *Quantity Imbalance*: Differences in the communication capabilities of IoT devices lead to variations in the scale of their traffic data, resulting in different sizes for each node's local dataset $|D_i|$. Similar to label imbalance, we utilize the Dirichlet distribution to allocate varying quantities of labeled samples to each node. The random vector $\mathbf{x}_n \in \mathbb{R}^N$ represents the distribution of sample quantities for each category across nodes, with each dimension of the vector indicating the proportion of samples assigned to different nodes. The vector

$\mathbf{x}_n$ is sampled from the distribution $P_n \sim \text{DP}(\alpha, G_0)$. The product $\mathbf{x}_n \times |D| = \{|D_i|\}_{i=1}^N$ corresponds to the local dataset size for each label at a node. This paper simultaneously employs the three aforementioned partitioning methods to divide the traffic dataset, thereby creating local datasets for experiments and achieving a Non-IID data distribution for IoT devices.

IV. THE PERFLID WORKFLOW

A. Architecture Design of Personalized Federated Meta-Learning Intrusion Detection

In the IoT scenario, the PFL framework for traffic intrusion detection is shown in Figure 1 and consists of three layers: the node layer, which includes IoT edge devices, security gateways, and firewalls; the data transmission layer, which mainly consists of communication base stations; and the central server layer, which is usually deployed on a cloud server. The cloud server is responsible for training and maintaining the global model M_g . After the training is completed, each device in the node layer then performs several rounds of local updates based on M_g to obtain the local personalized model $PM_l = \{pm_{l_0}, \dots, pm_{l_N}\}$. The specific implementation scheme of PerFLID is shown in Figure 2, and the final trained local personalized model PM_l can be integrated into the traffic intrusion detection tool. The final trained local personalized model PM_l can be integrated into a traffic intrusion detection tool to deploy effective traffic detection locally. We categorize data collection to model training into the following five steps:

- 1) To identify unknown network attacks, IoT end devices utilize the open-source tool CICflowmeter for traffic characterization. The tool samples traffic flowing through the gateway firewall, extracts feature information from packets stored in Packet Capture (PCAP) data format files and builds a local dataset in Comma Separated Values (CSV) format.
- 2) The central server employs the initialization parameter w_0 to train the initialization model, which is subsequently transmitted to the selected edge node via the IoT. The detection local model M_l of the edge nodes is kept the same as the central server. The task of this model is to learn traffic features and perform multi-class prediction. Considering that the traffic feature data collected by the edge devices exhibits uneven label distribution, feature distribution, and volume distribution, for local data adaptation, we use in Equation (2) as a loss function on the edge devices. This facilitates initial model training for local traffic feature updates and model parameters w_l .
- 3) Once an edge node has updated its model parameters through meta-learning, it uploads the new model parameters w_l to the central server via the base station. Subsequently, the central server performs a weighted aggregation based on the size of each node's data as weights, resulting in new parameters w_{l+1} for M_g .
- 4) The central server sends the aggregated parameter w_{l+1} to the selected edge nodes. The probability of each node being selected, determined by the gradient parameters of the local model M_l uploaded to the server, is calculated

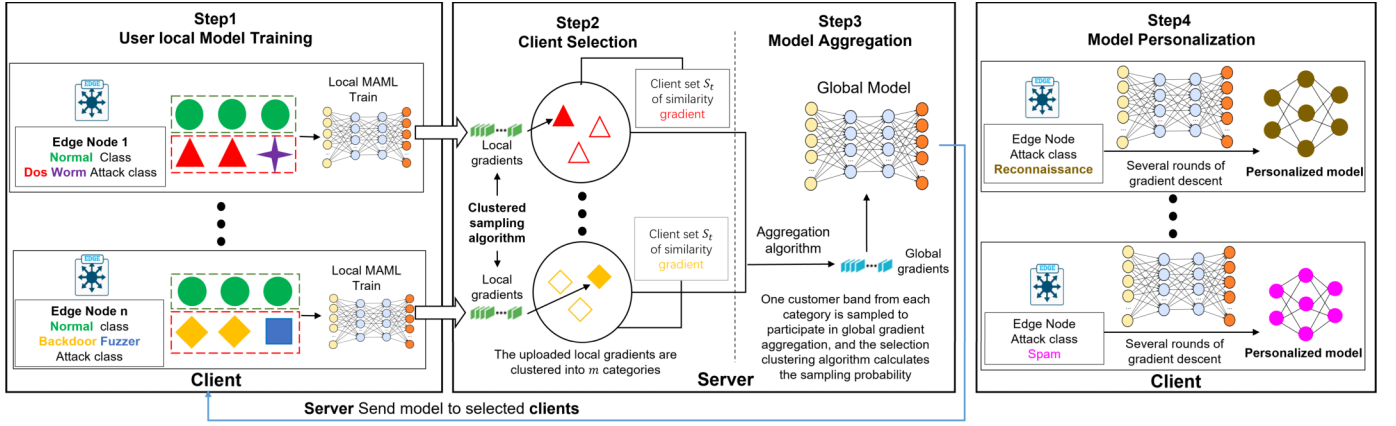


Fig. 2. The Overview of PerFLID Framework.

as shown in Algorithm 1. Then the edge node performs further meta-learning training and continues the iterative process described earlier.

- 5) After the last round of model aggregation, the global model M_g is distributed to all edge nodes. Subsequently, each edge node performs one or several rounds of training based on this model using the local dataset and finally obtains the personalized model PM_{l_i} .

B. Meta-Learning Training in Edge Node

Each device can ultimately train its personalized model based on the initial model to address the objective function in Equation (2) and enable the server to discern disparities between various edge node models. Algorithm 1 outlines the meta-learning training process. The initial step involves computing the gradient of each edge device's local loss function $\nabla F_i(w)$:

$$\nabla F_i(w) = (I - \alpha \nabla^2 f_i(w)) \nabla f_i(w - \alpha f_i(w)). \quad (3)$$

Calculating the gradient $\nabla f_i(w)$ in each round usually requires a high computational cost. Therefore, we adopt a batch of data D_i and conduct unbiased analysis $\tilde{\nabla} f_i(w, D_i)$:

$$\tilde{\nabla} f_i(w, D_i) := \frac{1}{|D_i|} \sum_{(x,y) \in D_i} \nabla l_i(w; x, y). \quad (4)$$

In this process, $l_i(w; x, y)$ represents the loss function of the edge device training model. In the second part, during the k th round of communication, the server initially selects the edge devices for the current global model w_k based on the node selection strategy. Each edge device $i \in S_k$ then executes τ steps of random gradient descent concerning F_i . These local updates generate a local sequence $w_{i,t}^{k+1} t = 0^\tau$, where $w_{i,0}^{k+1} = w_i^{k+1}$ and $w_{i,\tau}^{k+1} = w_i^{k+2}$ for $1 \leq t \leq \tau$:

$$w_{i,t}^{k+1} = w_{i,t-1}^{k+1} - \beta \tilde{\nabla} F_i(w_{i,t-1}^{k+1}), \quad (5)$$

S_k denotes the collection of nodes participating in the iteration. β represents the learning rate of local updates, where $k+1$ indicates training using the meta-model parameters of the k rounds. $\tilde{\nabla} F_i(w_{i,t-1}^{k+1})$ denotes the estimated value of $\nabla F_i(w_{i,t-1}^{k+1})$

in Equation (7). Then, adding both sides of Equation (5) for $1 \leq t \leq \tau$, we note that:

$$w_i^{k+2} = w_i^{k+1} - \beta \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^{k+1}). \quad (6)$$

In the final step, independent batches $D_{i,t}$, $D'_{i,t}$, and $D''_{i,t}$ are utilized to compute the stochastic gradient $\tilde{\nabla} F_i(w_{i,t-1}^{k+1})$ for all local iterations, as follows:

$$\begin{aligned} \tilde{\nabla} F_i(w_{i,t-1}^{k+1}) &:= (I - \alpha \tilde{\nabla}^2 f_i(w_{i,t-1}^{k+1}, D'_{i,t})) \\ &\tilde{\nabla} f_i(w_{i,t-1}^{k+1} - \alpha f_i(w_{i,t-1}^{k+1}, D_{i,t}), D'_{i,t}). \end{aligned} \quad (7)$$

Once the local update $w_{i,\tau}^{k+1}$ is completed, the edge node uploads it to the server. Subsequently, the server calculates the global w^{k+1} based on the aggregated weights p_i^k of each node:

$$w^{k+1} = \sum_{i \in S_k} p_i^k w_{i,\tau}^{k+1}. \quad (8)$$

Finally, the server distributes the global model w^{k+1} to the selected set of nodes S_k and initiates a new round of training. After a certain number of training rounds, each node locally updates its model M_l , ultimately obtaining a personalized model PM_l . Compared to M_l , the personalized model PM_l derived from our method better adapts to local data. We will theoretically and experimentally validate the effectiveness of our method in Sections V and VI, respectively.

C. Node Selection Based on Gradient Similarity Clustering

In FL, the global model is obtained by aggregating all nodes and performing a weighted summation of the local model parameters w_i^t returned by the nodes to the server,

$$w^t = \sum_{i \in N} p_i w_i^t. \quad (9)$$

Here, $p_i = \frac{|D_i|}{|D|}$ denotes aggregation weights of nodes participating in the iteration depend on the ratio of the number of local samples to the number of global samples. Since the FedAvg algorithm does not involve all nodes in each round of

Algorithm 1 PerFLID Workflow

Input: Initial iterate w_0 , each user local model gradient w_i , number of node samples $|D_i|$

Output: Global model parameters w^K , local personalization model PM_{l_i}

- 1: The server sends w_0 to all nodes, and the nodes perform one local gradient update to get w_i , which is uploaded to the server;
- 2: **for** $k : 0$ to $K - 1$ **do**
- 3: The server calculates the cosine similarity based on the saved node model parameters and performs hierarchical clustering, grouping them into m categories;
- 4: $t \leftarrow 1$ Start considering the first distribution $\{R_t(k)\}$;
- 5: $M \leftarrow 0$ Recalculate node selection probability;
- 6: **repeat**
- 7: **for** each node $i = 1$ to n **do**
- 8: **if** i in Current distribution $\{R_t(k)\}$ **then**
- 9: $r'_{k,i} = \frac{m+1}{2}|D_i|$, $M = M + r'_{k,i}$;
- 10: **else**
- 11: $r'_{k,i} = \epsilon \frac{|D_i|}{2}$, $M = M + r'_{k,i}$, $\epsilon \ll m$;
- 12: **end if**
- 13: Selection probabilities $r_{k,i} = \frac{r'_{k,i}}{M}$;
- 14: **end for**
- 15: Server generates distribution $\{R_t(k)\}$ based on $r_{k,i}$; $t \leftarrow t + 1$;
- 16: **until** $t > m$, distribution set $\{R_t(k)\}_{t=1}^m$;
- 17: The server selects m times according to distribution set $\{R_t(k)\}_{t=1}^m$, determine the selected node set S_k . The Server sends w^k to all nodes in S_k ;
- 18: **for** all $i \in S_k$ **do**
- 19: Set $w_{i,0}^k + 1 = w^k$;
- 20: **for** $t : 1$ to τ **do**
- 21: Compute the stochastic gradient $\tilde{\nabla} f_i(w_{i,t-1}^k + 1, D_i^t)$ using dataset D_i^t ;
- 22: Set $\tilde{w}_{i,t}^{k+1} = w_{i,t-1}^{k+1} - \alpha \tilde{\nabla} f_i(w_{i,t-1}^{k+1}, D_i^t)$;
- 23: Set $w_{i,t}^{k+1} = w_{i,t-1}^{k+1} - \beta (I - \alpha \tilde{\nabla}^2 f_i(w_{i,t-1}^{k+1}, D_i^t)) \tilde{\nabla} f_i(w_{i,t-1}^{k+1}, D_i^t)$;
- 24: **end for**
- 25: node i sends $w_{i,\tau}^{k+1}$ back to server;
- 26: **end for**
- 27: Server updates its model by averaging over selected models;
- 28: **end for**
- 29: Nodes perform several rounds of gradient updates locally based on w^K to get PM_{l_i} ;
- 30: **return:** Initialize global model parameters w^K , local personalization model PM_{l_i} .

aggregation, but randomly selects $m = rN$ nodes based on the ratio r , the final global model with weights is:

$$w^t = \sum_{i \in S_k} p_i w_i^t + \sum_{i \notin S_k} p_i w_i^{t-1}. \quad (10)$$

In Equation (10), the set of nodes S_k is randomly selected by the server. However, this random selection strategy poses potential security risks in real-world federated intrusion detection scenarios. If a malicious node exists, it may be randomly

selected to join the set S_k and participate in the aggregation process through backdoor attacks. Once a node is subjected to an untargeted backdoor attack, the task accuracy of the global model is compromised [32]. Under such influence, the global model may ignore the features of some key nodes, worsening the convergence effect of the global model. Consequently, the generalization performance of the final trained initial model will be affected, hindering the process of user-personalized model training.

Several algorithms have been proposed to address the problem of backdoor attacks in federated learning models, including distance-based defense methods [33] and defense methods based on Singular Value Decomposition (SVD) and clustering [34]. Our proposed aggregation method is similar to [34], however, our paper places a greater emphasis on designing a new node selection strategy for the rapid aggregation of local personalized models. This strategy must be unbiased and ensure that the total expected value of each selected node equals the global aggregation value obtained by selecting all nodes, as shown in the following equation:

$$\mathbb{E}_{S_k} [w^t] = \mathbb{E}_{S_k} \left[\sum_{j \in S_k} p_j w_j^t \right] := \sum_{i \in N} p_i w_i^t. \quad (11)$$

In Algorithm 1, the server retains the model parameters of edge nodes from the previous round of model communication. We assume that the model parameters are represented as vectors. Many studies have explored node clustering in client selection, utilizing various clustering algorithms like DBSCAN [35], hierarchical clustering [26], and K-means [36]. Due to the large size of local models and the presence of high-dimensional features in the models, some have proposed their distance metrics [33]. In our approach, we measure the similarity of node models using cosine similarity and then classify the node models using Arccos and hierarchical clustering. The server then measures the similarity between each pair of client model parameters using the cosine similarity, which is calculated as follows:

$$\text{cosine_similarity}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}. \quad (12)$$

where $A \cdot B$ is the dot product of vectors A and B , and $\|A\|$ and $\|B\|$ are the norms of vectors A and B , respectively. Next, the cosine similarity is converted to an angle using the inverse cosine function (Arccos) to better represent the differences between the vectors, which is calculated as:

$$\theta = \arccos(\text{cosine_similarity}(A, B)). \quad (13)$$

where θ is the angle between the vectors A and B . Then calculate the angle between the gradients of each node to get the angle matrix Θ . Finally, select a clustering algorithm (such as K-means, hierarchical clustering, etc.) to cluster the angle matrix Θ and obtain m categories.

Note that the number of clusters should match the number of nodes sampled by the server to ensure unbiased selection. Subsequently, the probability of each node being selected for each selection is calculated based on the number of samples in

the cluster, generating m independent probability distributions $\{R_i(k)\}_{i=1}^m$.

Although our gradient clustering-based node selection method is not specifically designed for backdoor attack scenarios, it can still screen out malicious nodes by leveraging the gradient differences between nodes. This capability enables it to be combined with other secure aggregation methods to defend against backdoor attacks in each round of federated learning, thereby circumventing security risks during the training of personalized NIDS models.

We will prove our algorithm satisfies the assumption of unbiased selection in Section V. Although Algorithm 1 requires execution in each round of communication, it does not incur additional communication overhead, and our experimental results in Section VI also demonstrate a faster convergence during training.

V. CONVERGENCE ANALYSIS OF PERFLID

In this section, we provide a theoretical analysis of how PerFLID improves the convergence speed of federated learning. This includes the model assumptions used to analyze the convergence of federated meta-learning in Section V-A, as well as an analysis of how meta-learning with randomly participating nodes in aggregation affects convergence, as discussed in Section V-B. Furthermore, we present a theoretical examination of the strategy for selecting sets for updates based on node gradient clustering in each training round. We demonstrate that this strategy leads to a more effective lower bound for aggregation selection, as shown in Section V-C and Section V-D.

A. Modeling Assumptions

We concentrate on non-convex settings and delineate the communication between the server and nodes in two consecutive rounds to ensure that the anticipated decrease in the global loss function satisfies $\mathbb{E}[F(w^{k+1})] - \mathbb{E}[F(w^k)] \leq \varepsilon$. For the theoretical analysis of PerFLID, several standard assumptions are typically made on the FML models (refer to e.g., [28], [37], [38]). We will utilize the following assumptions in our analysis:

Assumption 1: The gradient of functions f_i is bounded by a nonnegative constant U_i , i.e., $\|\nabla f_i(w)\| \leq U_i$

Assumption 2: Function f_i is twice continuously differentiable and L_i -smooth for every node $i \in \{1, \dots, N\}$, i.e., $\|\nabla f_i(w) - \nabla f_i(u)\| \leq L_i \|w - u\| \forall w, u \in \mathbb{R}^d$. The gradient Lipschitz assumption also implies that f_i satisfies the following conditions for all $w, u \in \mathbb{R}^d$:

$$|f_i(w) - f_i(u) - \langle \nabla f_i(u), (w - u) \rangle| \leq \frac{L_i}{2} \|w - u\|^2. \quad (14)$$

As discussed in Section IV-B, the update rules in Algorithm 1 involve the second derivatives of all nodes' loss functions. Therefore, it is necessary to impose positive definiteness assumptions on the Hessian matrices of all f_i .

Assumption 3: For every node $i \in \{1, \dots, N\}$, the Hessian of function f_i is ρ_i -Lipschitz continuous, i.e.,

$$\|\nabla^2 f_i(w) - \nabla^2 f_i(u)\| \leq \rho_i \|w - u\| \forall w, u \in \mathbb{R}^d. \quad (15)$$

A significant disparity in loss between nodes and the server can hinder model convergence in each communication round. Therefore, we make the following similar assumptions about their gradients.

Assumption 4: The gradient of local functions $F_i(w)$ is at most B -locally dissimilar from $F(w)$ for each node i , i.e., $\|\nabla F_i(w)\| \leq B_i \|\nabla F(w)\|$. We expect both sides of the equation to get $E_i[\|\nabla F_i(w)\|^2] \leq \|\nabla F(w)\|^2 B^2$.

In the remainder of this paper, we define $\rho := \max \rho_i$ and $L := \max L_i$, which can be regarded as bounds in each assumption.

Assumption 5: The variance of the difference between the local and global model parameters is limited for all nodes i at any local update time t , i.e.,

$$\mathbb{E} \left[\frac{1}{n} \sum_{i=1}^n \|w_{i,t}^k - w_t^k\|^2 \right] \leq \beta^2 t \tau \sigma_w^2 \quad \text{for any } 0 \leq t \leq \tau. \quad (16)$$

The discrepancies in gradients and Hessians between nodes and the server during each training round are influenced by the participating training datasets D_i, D'_i , and D''_i . We make assumptions about the differences in gradients and Hessians.

Assumption 6: The gradient estimate $\widetilde{\nabla} F_i(w)$ in (7) which is computing by using D_i, D'_i and D''_i is close enough to the true value, i.e.,

$$\begin{aligned} \mathbb{E} [\|\widetilde{\nabla} F_i(w) - \nabla F_i(w)\|] &\leq \sigma_{\varphi_D}^2 := \varphi(D_i), \\ \mathbb{E} [\|\widetilde{\nabla} F_i(w) - \nabla F_i(w)\|^2] &\leq \sigma_{\phi_D}^2 := \phi(D_i, D'_i, D''_i). \end{aligned} \quad (17)$$

B. Meta-Learning With Random Node Selection

As discussed in Section IV-B, when there are numerous nodes, the standard federated meta-learning algorithm randomly selects m nodes as set S_k to perform local training in each communication round. The enhancement of the global model by nodes in each round of updates is influenced by the distribution of each node's data. Therefore, random selection cannot consistently pick the nodes that contribute optimally to global updates. To prove this point, we first need to establish the intermediate result that, under Algorithm Assumptions 1 to 3, the local meta-function $F_i(w)$ defined by Equation (2) and the average of each node's meta-function $F(w) = (1/N) \sum_{i=1}^N F_i(w)$ are both smooth.

Lemma 1: If Assumptions 1, 2, and 3 hold, then F_i in Equation (2) with $\alpha \in [0, 1/L]$ is smooth with parameter $L := 4L + \alpha\beta U$. Hence, the $F(w)$ is also smooth with parameter L .

Second, we use the B-locally similarity in Assumptions 4 to derive the difference between the local loss $F_i(w)$ and the global $F(w)$.

Lemma 2: When B -locally dissimilar in Assumptions 4 follows that: $B \leq \sqrt{1 + \sigma_B^2}$. Then the true rather than estimated bounded variance in Assumptions 6 holds, i.e., $E_i[\|\nabla F_i(w) - \nabla F(w)\|^2] \leq \sigma_B^2$. Using Lemmas 1, and 2, we analyze the expected reduction of the target when executing a step of Algorithm 1 with nodes randomly selected.

Theorem 1: Consider the objective function F defined in Equation (1) for the case that $\alpha \in [0, 1/L]$. Suppose that the conditions in Assumptions 1 to 4 and 6 are satisfied, and recall the definitions of L from Lemma 1. Consider running Algorithm 1 with the randomly selected set S_k for K rounds with τ local updates in each round and with $\beta \leq 1/(10\tau L)$. The expected decrease in the global loss function satisfies

$$\begin{aligned} F(w^{k+1}) &\leq F(w^k) - \frac{3\tau\beta}{10} \|\nabla F(w^k)\|^2 \\ &\quad + \tau\beta[(2L\beta\sigma_{\phi_D}^2 - \sigma_{\phi_D}^2) + 2(2L\beta - 1)\sigma_B^2] \\ &\quad + \tau\beta[2(\tau - 1)L^2\beta^2(L\beta - 1)]. \end{aligned} \quad (18)$$

Proof: We define the average of randomly selected nodes in the k -th round of updates as $\bar{w}^k = 1/m \sum_{S_k} w_i^k$. By Lemma 1, we know $F(\bar{w})$ is smooth therefore by Assumption 2, $F(\bar{w})$ also satisfies the radiant Lipschitz assumption, thus we have

$$\begin{aligned} F(\bar{w}^{k+1}) &\leq F(\bar{w}^k) + \langle \nabla F(\bar{w}^k), \bar{w}^{k+1} - \bar{w}^k \rangle \\ &\quad + \frac{L}{2} \|\bar{w}^{k+1} - \bar{w}^k\|^2, \end{aligned} \quad (19)$$

recall the equality (6)

$$w_i^{k+1} - w^k = -\beta \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k), \quad (20)$$

due to

Assumption 5, for the nodes selected to participate in the aggregation, sum the two sides of the equation (20) separately, we obtain

$$\bar{w}^{k+1} - \bar{w}^k = -\frac{\beta}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \quad (21a)$$

$$\|\bar{w}^{k+1} - \bar{w}^k\|^2 = \left\| \frac{\beta}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\|^2. \quad (21b)$$

Put (21a), (21b) into (19) and take expectation from both sides

$$\begin{aligned} \mathbb{E}[F(\bar{w}^{k+1})] &\leq \mathbb{E}[F(\bar{w}^k)] \\ &\quad - \beta \mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\rangle \right] \\ &\quad + \frac{L}{2} \beta^2 \mathbb{E} \left[\left\| \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\|^2 \right]. \end{aligned} \quad (22)$$

Next, note that

$$\frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k)$$

$$= A + B + C + \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \nabla F_i(\bar{w}_{i,t-1}^k), \quad (23)$$

We will separately bound the terms in the above formula in the Supplementary Material, and thus, by Equations (58), (60), (62) and (63), we have

$$\begin{aligned} \mathbb{E} \left\| \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\|^2 &\leq 4\tau \mathbb{E} [\|\nabla F(w_{i,t-1}^k)\|^2] \\ &\quad + 4\tau\sigma_{\phi_D}^2 + 8\tau\sigma_B^2 + 4\tau(\tau - 1)\sigma_w^2 L^2 \beta^2. \end{aligned} \quad (24)$$

Now, we lower bound the term

$$\mathbb{E} \left[\left\langle \nabla F(w^k), \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\rangle \right]. \quad (25)$$

We will separately bound the terms in the above formula in the Supplementary Material, by (54), we have

$$\begin{aligned} &\mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\rangle \right] \\ &= \mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \left(A + B + C + \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \nabla F_i(\bar{w}_{i,t-1}^k) \right) \right\rangle \right] \\ &\geq \mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \nabla F_i(\bar{w}_{i,t-1}^k) \right\rangle \right] \\ &\quad - \|\mathbb{E}[\langle \nabla F(\bar{w}^k), A \rangle]\| - \|\mathbb{E}[\langle \nabla F(\bar{w}^k), B + C \rangle]\|. \end{aligned} \quad (26)$$

By plugging Equations (68) to (70) in Equation (26) implies

$$\begin{aligned} &\mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\rangle \right] \\ &\geq \frac{\tau}{2} \|\nabla F(w^k)\|^2 - \tau\sigma_{\phi_D}^2 - 2\tau\sigma_B^2 - 2\tau(\tau - 1)L^2\beta^2\sigma_w^2. \end{aligned} \quad (27)$$

Due to randomly selecting nodes to participate in aggregation, we have $\mathbb{E}_{S_k}[F(\bar{w}^k)] = F(w^k)$. Substituting Equation (27) and Equation (25) in Equation (22) implies

$$\begin{aligned} F(w^{k+1}) &\leq F(w^k) - \tau\beta \left(\frac{1}{2} - 2L\beta \right) \|\nabla F(w^k)\|^2 \\ &\quad + \tau\beta(2L\beta\sigma_{\phi_D}^2 + \sigma_{\phi_D}^2) + 2\tau\beta\sigma_B^2(2L\beta + 1) \\ &\quad + 2\tau(\tau - 1)L^2\beta^3\sigma_w^2(L\beta + 1) \\ &\leq F(w^k) - \frac{3\tau\beta}{10} \|\nabla F(w^k)\|^2 \\ &\quad + \tau\beta[(2L\beta\sigma_{\phi_D}^2 + \sigma_{\phi_D}^2) + 2\sigma_B^2(2L\beta + 1)] \\ &\quad + \tau\beta[2(\tau - 1)L^2\beta^2(L\beta + 1)]. \end{aligned} \quad (28)$$

The last inequality is obtained using $\beta \leq 1/(10\tau L)$, which gives us the desired result. $\square$

Theorem 1 shows a dependency on parameters L, B, β, τ of the FML model. Note that $\sigma_{\phi_D}^2, \sigma_{\phi_D}^2$ is not a constant, and as expressed in Assumption 5, we can make it arbitrarily small by choosing batch sizes D, D' and D'' large enough. Once again, we focus on special cases to examine the tightness of our results. Let $\sigma_{\phi_D}^2 = \sigma_{\phi_D}^2 = 0$, and $\tau = 1$. In this case, reduces to stochastic gradient descent, where the only source of stochasticity is the batches of gradient. The second term on the right-hand side in Equation (18) reduces to $\mathcal{O}(\beta^2 L B^2)$.

This is the classic result for stochastic gradient descent for nonconvex functions, and we recover the lower bounds [39].

C. Clustering-Based Node Selection

Algorithm 1 states that the node with the highest contribution to the global model in each communication round is most probably selected as the solution to the gradient difference between nodes. The contribution of each node to the global model is represented by hierarchically clustering the gradients uploaded by the nodes and generating independent distributions $\{R_t(k)\}_{t=1}^m$. Selection based on the clustering results can speed up the convergence of the model compared to randomly selecting nodes. This point is supported by [40] theoretical proof of convergence of the Multinomial Distribution (MD) Selection section. First, we give Proposition V.1.

Proposition 1: The probability $r_{k,i}^t$ of clustered selection from the distribution $\{R_t(k)\}_{t=1}^m$ in Proposition 1 satisfies Equation (29) the following property as a sufficient condition for unbiased selection:

$$\begin{aligned} \forall k \in \{1, \dots, m\}, \quad \sum_{i=1}^N r_{k,i}^t &= 1, \\ \forall i \in \{1, \dots, n\}, \quad \sum_{t=1}^m r_{k,i}^t &= m \frac{|D_i|}{|D|}. \end{aligned} \quad (29)$$

Proof: By Equation (29), when selecting one node from one of the m distributions $R_t(k)$, we have:

$$\mathbb{E}_{R_t(k)} \left[\sum_{j \in R_t(k)} p_j(R_t(k)) w_j^k \right] = \sum_{i=1}^n r_{k,i}^t w_i^k.$$

Due to the linear relationship of the expected values, the expected new global model is the average of the weighted models obtained for each distribution $\{R_t(k)\}_{t=1}^m$ according to Equation (30).

$$\mathbb{E}_{S_k} [w^k] = \sum_{k=1}^m \frac{1}{m} \sum_{i=1}^n r_{k,i}^t w_i^k = \sum_{i=1}^n p_i w_i^k, \quad (30)$$

which means that our theory is valid. $\square$

We demonstrate our cluster selection, as described in Proposition V.1, satisfies Lemma V.3. Additionally, the MD selection method has similar bounds, indicating that using the FedAvg aggregation algorithm exhibits identical asymptotic behavior when using our proposed clustering and MD selection.

Lemma 3: Suppose $C_1, C_2, \dots, C_m$ are the index of sampled nodes. We have

$$\begin{aligned} \mathbb{E}_{S_k} \left[\frac{1}{m} \sum_{j=1}^m w_{C_j} \right] &= \sum_{i=1}^n \frac{|D_i|}{|D|} w_i, \\ \mathbb{E} \left[\left\| \frac{1}{m} \sum_{i \in S_k} w_i^k - w^k \right\|^2 \right] &\leq \sum_{i=1}^n p_i \|w_i^k - w^k\|^2. \end{aligned} \quad (31)$$

We will analyze the convergence of node-sampled federated meta-learning in the next subsection.

D. Meta-Learning With Clustering-Based Node Selection

In Theorem V.1 we prove an upper bound on the convergence of meta-learning for random node selection satisfying Assumptions 1 to 3. In Proposition 1 we show that node selection based on cluster sampling has the same bound on FL convergence as MD-based selection. We now combine the two conclusions to prove an upper bound on the convergence of the node selection strategy using meta-learning with cluster-based in Theorem V.2:

Theorem 2: Consider the objective function F defined in (1) for the case that $\alpha \in [0, 1/L]$. Suppose that the conditions in Assumptions 1–5 are satisfied, and recall the definitions of L from Lemma 1. Consider running Algorithm 1 along with its node selection strategy. Then obtain the selected set S_k for K rounds with τ local updates in each round and with $\beta \leq 1/(10\tau L)$. We have the following expected decrease in the global objective

$$\begin{aligned} F(w^{k+1}) &\leq F(w^k) - \frac{3\tau\beta}{10} \|\nabla F(w^k)\|^2 \\ &\quad + \tau\beta [(2L\beta\sigma_{\phi_D}^2 - \sigma_{\phi_D}^2) + 2p(2L\beta - 1)\sigma_B^2] \\ &\quad + \tau\beta p [2(\tau - 1)L^2\beta^2(L\beta - 1)]. \end{aligned} \quad (32)$$

Proof: We now prove the convergence of Algorithm 1, FML using our node selection strategy. Similarly, We define the average of selected nodes in the k -th round of updates as $\bar{w}^k = 1/m \sum_{i \in S'_k} w_i^k$. To distinguish it from the set of randomly chosen nodes, according to Algorithm 1, we denote by S'_k the set of nodes chosen and $\mathbb{E}_{S'_k}[F(\bar{w}^k)]$ as the expectation of the global loss, for the subsequent proofs. By Lemma 1 and Equation (22), we have

$$\begin{aligned} \mathbb{E}_{S'_k} [F(\bar{w}^{k+1})] &\leq \mathbb{E}_{S'_k} [F(\bar{w}^k)] \\ &\quad - \beta \mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \frac{1}{m} \sum_{i \in S'_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\rangle \right] \\ &\quad + \frac{L}{2} \beta^2 \mathbb{E} \left[\left\| \frac{1}{m} \sum_{i \in S'_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\|^2 \right]. \end{aligned} \quad (33)$$

We will prove Theorem V.2 along the same lines as we proved Theorem V.1. Note that

$$\begin{aligned} &\frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \\ &= A' + B' + C' + \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \nabla F_i(\bar{w}_{t-1}^k). \end{aligned} \quad (34)$$

Based on our assumptions and Lemma 2, the upper bounds of A' and B' remain constant in the above equation. Next, we demonstrate the impact of the choice strategy on C' .

$$\begin{aligned} \|C'\|^2 &\leq \sum_{t=1}^{\tau} \left(\frac{1}{m} \sum_{i \in S'_k} \|\nabla F_i(w_{i,t-1}^k) - \nabla F_i(\bar{w}_{t-1}^k)\|^2 \right) \\ &\leq \frac{L^2}{m} \sum_{t=1}^{\tau} \sum_{i \in S'_k} \|w_{i,t-1}^k - \bar{w}_{t-1}^k\|^2 \end{aligned}$$

$$= L^2 \sum_{t=1}^{\tau} \|w_{t-1}^k - \bar{w}_{t-1}^k\|^2. \quad (35)$$

Based on the conclusion in Lemma 3 to bound C' , taking the mathematical expectation for both sides of Equation (35).

$$\begin{aligned} \mathbb{E}[\|C'\|^2] &\leq L^2 \mathbb{E} \left[\sum_{t=1}^{\tau} \mathbb{E} \|w_{t-1}^k - \bar{w}_{t-1}^k\|^2 \right] \\ &= L^2 \sum_{t=1}^{\tau} \left[\mathbb{E} \|w_{t-1}^k - w_{t-1}^k\|^2 \right] \\ &\leq L^2 \sum_{t=1}^{\tau} \sum_{i=1}^n p_i \|w_{i,t-1}^k - w_{t-1}^k\|^2 \\ &\leq pL^2 \beta^2 \tau(\tau-1) \sigma_w^2. \end{aligned} \quad (36)$$

Then, we bound the last term of Equation (34), and we have

$$\mathbb{E} \left[\left\| \frac{1}{m} \sum_{i \in S_k} \nabla F_i(\bar{w}_{t-1}^k) \right\|^2 \right] \leq \mathbb{E} [\|\nabla F(w_{t-1}^k)\|^2] + p\sigma_B^2. \quad (37)$$

Note that, by Equation (64), we have

$$\begin{aligned} \left\| \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\|^2 &\leq 4\|A'\|^2 + 4\|B'\|^2 + 4\|C'\|^2 \\ &\quad + 4 \left\| \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \nabla F_i(\bar{w}_{t-1}^k) \right\|^2, \end{aligned} \quad (38)$$

and thus, by Equations (58), (60), (62) and (63), we have

$$\begin{aligned} \mathbb{E} \left[\left\| \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\|^2 \right] &\leq 4\tau \mathbb{E} [\|\nabla F(w^k)\|^2] \\ &\quad + 4\tau \sigma_\phi^2 + 4\tau(1+p)\sigma_B^2 + 4L^2 \beta^2 \tau(\tau-1) \sigma_w^2. \end{aligned} \quad (39)$$

Next, we lower bound the term

$$\mathbb{E} \left[\left\langle \nabla F(w^k), \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\rangle \right]. \quad (40)$$

By (34), we have

$$\begin{aligned} &\mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \frac{1}{m} \sum_{i \in S'_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\rangle \right] \\ &\geq \mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \frac{1}{m} \sum_{i \in S'_k} \sum_{t=1}^{\tau} \nabla F_i(\bar{w}_{i,t-1}^k) \right\rangle \right] \\ &\quad - \mathbb{E} [\langle \nabla F(\bar{w}^k), A' \rangle] - \mathbb{E} [\langle \nabla F(\bar{w}^k), B' + C' \rangle]. \end{aligned} \quad (41)$$

We bound the terms of the above inequality separately. First note that

$$\begin{aligned} &\mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \frac{1}{m} \sum_{i \in S'_k} \sum_{t=1}^{\tau} \nabla F_i(\bar{w}_{i,t-1}^k) \right\rangle \right] \\ &= \tau \mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \mathbb{E} \left[\frac{1}{m} \sum_{i \in S'_k} \nabla F_i(\bar{w}_i^k) \right] \right\rangle \right] \\ &= \tau \mathbb{E} \|\nabla F(w^k)\|^2. \end{aligned} \quad (42)$$

Second, we have

$$\begin{aligned} \mathbb{E} [\langle \nabla F(\bar{w}^k), A' \rangle] &= \mathbb{E} [\langle \nabla F(\bar{w}^k), \mathbb{E} [A'] \rangle] \\ &\leq \frac{\tau}{4} \|\nabla F(w^k)\|^2 + \tau \sigma_{\varphi_D}^2, \end{aligned} \quad (43)$$

where the last inequality follows from

Assumption 5. Last, we use the fact that

$$\begin{aligned} \mathbb{E} [\langle \nabla F(\bar{w}^k), B' + C' \rangle] &\leq \frac{\tau}{4} \mathbb{E} [\|\nabla F(w^k)\|^2] \\ &\quad + 2E [\|B'\|^2] + 2E [\|C'\|^2]. \end{aligned} \quad (44)$$

Plugging Equations (68) to (70) in Equation (26) implies

$$\begin{aligned} &\mathbb{E} \left[\left\langle \nabla F(\bar{w}^k), \frac{1}{m} \sum_{i \in S_k} \sum_{t=1}^{\tau} \tilde{\nabla} F_i(w_{i,t-1}^k) \right\rangle \right] \\ &\geq \frac{\tau}{2} \|\nabla F(w^k)\|^2 - \tau \sigma_{\varphi_D}^2 - 2\tau \sigma_B^2 - 2pL^2 \beta^2 \tau(\tau-1) \sigma_w^2. \end{aligned} \quad (45)$$

Due to randomly selecting nodes to participate in aggregation, we have $\mathbb{E}_{S_k} [F(\bar{w}^k)] = F(w^k)$. Substituting Equation (27) and Equation (25) in Equation (22) implies

$$\begin{aligned} \mathbb{E}_{S'_k} [F(w^{k+1})] &\leq F(w^k) - \tau\beta \left(\frac{1}{2} - 2L\beta \right) \|\nabla F(w^k)\|^2 \\ &\quad + \tau\beta(2L\beta\sigma_{\phi_D}^2 + \sigma_{\varphi_D}^2) + 2\tau\beta\sigma_B^2(2L\beta + 1) \\ &\quad + 2\tau(\tau-1)L^2\beta^3\sigma_w^2(L\beta + 1) \\ &\leq F(w^k) - \frac{3\tau\beta}{10} \|\nabla F(w^k)\|^2 \\ &\quad + \tau\beta[(2L\beta\sigma_{\phi_D}^2 + \sigma_{\varphi_D}^2) + 2p(2L\beta + 1)\sigma_B^2] \\ &\quad + \tau\beta p[2(\tau-1)L^2\beta^2(L\beta + 1)]. \end{aligned} \quad (46)$$

The last inequality is obtained using $\beta \leq 1/(10\tau L)$, which gives us the desired result. $\square$

Theorem V.2 differs from Theorem V.1 in that it concludes that the upper bound on convergence decreases each round with the addition of client selection strategies. This bound is now limited to the maximum probability of being selected, p . In general, $p \leq 1/m < 1$, which inevitably results in a lower upper bound on convergence, thus speeding up the convergence of the model during training. Let $\sigma_{\varphi_D}^2 = \sigma_{\phi_D}^2 = 0$, and $\tau = 1$. In this case, the second term on the right-hand side in Equation (32) reduces to $\mathcal{O}(p\beta^2 L B^2)$, which is similar to complexity in Theorem V.1, but with a new constraint term.

VI. EVALUATION

A. Experimental Settings and Datasets

1) *Environmental Setup:* In this study, both our global and local models utilize three different deep learning neural networks for three different datasets.¹ Note that the model

¹The CNN for UNSW-NB15 dataset has 7 layers with the following structure: $3 \times 3 \times 16$ Convolutional $\rightarrow 2 \times 2$ MaxPool $\rightarrow 3 \times 3 \times 32$ Convolutional $\rightarrow 2 \times 2$ MaxPool $3 \times 3 \times 64$ Convolutional $\rightarrow 2 \times 2$ MaxPool $\rightarrow z \times 11$ Fully connected. The LSTM for BoT-IoT dataset has 6 layers with the following structure: $(z, 15)$ LSTM $\rightarrow 20 \times 60$ Fully connected $\rightarrow 60 \times 80$ Fully connected $\rightarrow 80 \times 90$ Fully connected $\rightarrow 90 \times 4 \rightarrow$ softmax, where z depends on the number of input features. The CNN-BiLSTM for CIC-IDS2017 has 11 layers and its structure is in the link: <https://github.com/fisher85/ml-cybersecurity>

structure was adjusted to fit the various datasets while maintaining uniformity across different comparison methods. The model and the PFL framework were implemented using PyTorch 1.13.1. Experiments were conducted on the Ubuntu 20.04.3 LTS platform with an NVIDIA GeForce RTX 2080TI GPU.

2) *Dataset Description and Preprocessing*: In this paper, we conducted experiments using three widely-used IoT traffic intrusion detection datasets: the UNSW-NB15 dataset [41], the CIC-IDS2017 dataset [42], and the BoT-IoT dataset [43]. The UNSW-NB15 dataset was generated by the UNSW Cyber Range Laboratory, combining real-world benign traffic with modern, synthetic attack behaviors. The CIC-IDS2017 dataset includes both benign and various common attack types, providing raw traffic data as well as network traffic analysis results, which include timestamps, source and destination IPs, ports, protocols, and attack labels. The BoT-IoT dataset contains over 72 million records, covering a wide range of attacks including DDoS, DoS, OS scanning, service scanning, keystroke logging, and data exfiltration. These datasets have been widely used to evaluate intrusion detection systems, and their generation methods are well-documented in the respective publications, ensuring a diverse and representative set of attack scenarios and network conditions. Our preprocessing of this three dataset involves three steps: (1) conducting dimensionality reduction (PCA) on the three-column feature variables of character types and applying one-hot encoding to ensure uniqueness; (2) segregating the merged character variables and other features from the label class; (3) filling in missing data and performing reselection and feature normalization operations according to the Non-IID settings.

3) *Baselines*: We selected four state-of-the-art federated intrusion detection models: Deepfed [16], FSLAD [21], Fed-ANIDS [44], and SSFL [45] as baselines. These methods showcase significant advancements in federated learning for intrusion detection over the past four years. Our method, PerFLID, is designed to adapt to local data distributions. Additionally, unlike these models, our approach introduces a novel node selection mechanism to accelerate federated learning.

4) *Dataset Partitioning*: We utilize the Non-IID data partitioning strategy mentioned in Section III-B to segment the three datasets and create local traffic datasets for each node. The specific process is as follows: we allocate 80% of the dataset for training nodes, reserving the remaining 20% for testing nodes. Regardless of being a training or testing node, the dataset adheres to imbalances in attack type distribution, data characteristics, and quantity, aligning with the heterogeneity of real IoT data. Using the UNSW-NB15 dataset as an example, this dataset includes nine types of attacks, forming nine labels. We divide these labels among a specified number of nodes based on a Dirichlet distribution, ensuring that each label's probability for a node is a sample from the Dirichlet distribution. Then, we calculate the number of attack types each node contains based on this probability (excluding attack types with fewer than 5 samples). Finally, we ensure each node has at least one attack type but no more than five. When the number of training nodes = 100, at a certain node, the major

attack types (including normal traffic) and their sample counts are as follows: Normal 637 (63.89%), Backdoor 16 (1.61%), DoS 163 (16.35%), Exploits 86 (8.63%), Reconnaissance 88 (8.83%), Worms 7 (0.70%). For each round of experiments, training and test data are randomly selected based on the aforementioned ratio, with the experiment repeated ten times, and the average value serving as the experimental result.

5) *Evaluation Metrics*: Given that each edge node in our PerFLID model may encounter n (where $n \geq 3$) unknown types of attacks, we evaluate the performance of our model using accuracy, false alarm rate, and macro-F1 score as evaluation metrics. The calculation of these metrics is as follows:

Accuracy rate: the proportion of correctly classified traffic samples (including normal traffic and abnormal traffic) to the total samples.

$$\text{Accuracy} = \frac{T_P + T_N}{T_P + T_N + F_P + F_N} = \frac{1}{N} \sum_{i \in N} \frac{T_{P_i} + T_{N_i}}{|D_i|}. \quad (47)$$

Recall: the true positive rate, represents the proportion of normal traffic samples incorrectly identified as abnormal among all samples.

$$\text{Recall} = \frac{T_P}{T_P + F_N}. \quad (48)$$

Macro F1-score: Considering the large difference in the number of attack samples for each type of traffic at edge nodes, calculate the Macro F1-score. The calculation formula for the Macro F1-score is:

$$\text{Macro F1-score} = \frac{1}{K} \sum_{i=1}^K 2 \cdot \frac{\text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}. \quad (49)$$

where K is the total number of classes, The Macro F1-score measures the performance of a multi-class classification model by averaging the F1-scores of each class.

B. The Performance of Personalization in PerFLID

Capability testing of different federated learning distribution parameter models involves allocating training data based on the three imbalances discussed in Section III-B. The allocation method utilizes the label Dirichlet selection technique, where traffic data labels are distributed according to the parameter α , controlling the level of data heterogeneity. For all three datasets, we set the same parameters α to [0.1, 0.5, 1.0, 5, 10]. Similarly, the test node's data follows the distribution with the same parameters. By training our model under different parameters, we aim to evaluate the model's accuracy when tested on various edge nodes. The results are illustrated in Figure 3 and according to Figure 3, we evaluate the personalization performance from accuracy:

1) *Accuracy*: Personalized traffic intrusion detection models are expected to demonstrate high accuracy in local detection tasks [19]. We measure the degree of personalization by the average improvement in the classification accuracy of unknown types of traffic of the edge node's personalized model before and after training. According to Figure 3a, where $\alpha = 0.1$ represents the highest data heterogeneity, PerFLID achieved the most significant personalization improvement of

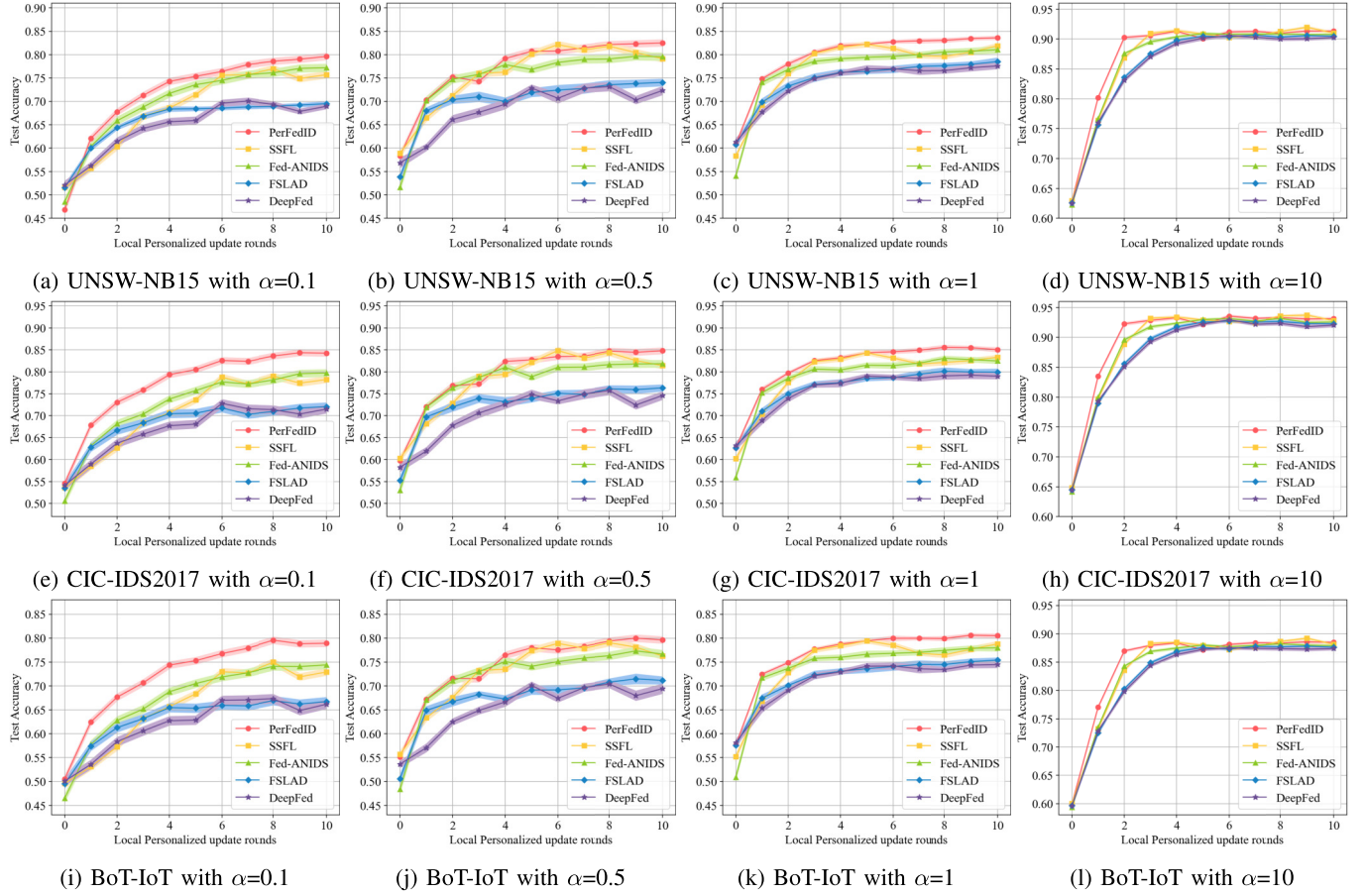


Fig. 3. Comparison results of the model accuracy in detecting unknown attacks with varied data distribution variance parameters α on UNSW-NB15, CIC-IDS2017, BoT-IoT datasets under localized personalized updates. (Since multiple nodes were tested, the shaded portions of the graph represent confidence intervals for the accuracy, which is the same in Figure 4).

34.5%, compared to 30.1% with Fed-ANIDS and 18.7% with DeepFed models. Table I presents specific experimental results, comparing *precision*, *recall*, and *macro-F1* scores obtained by PerFLID, FSLAD, Fed-ANIDS, SSFL, and DeepFed models when federated training to meet various personalized needs. Notably, at $\alpha = 0.1$, our method achieves *precision*, *recall*, and *macro-F1* scores of 79.6%, 74.5%, and 75.5%, respectively, on the UNSW-NB15 dataset, 82.5%, 77.4%, and 78.7% on the CIC-IDS2017 dataset, and 90.5% 89.9% 90.3% on the Bot-IoT dataset.

Several observations can be made from Figure 3: First, for low personalization needs ($\alpha = 10$), all federated learning models achieved 90% classification accuracy, as shown in Figure 3d. However, only PerFLID performed well under strong personalization requirements ($\alpha = 0.1$), achieving 79.6% accuracy in classifying unknown attacks using the CNN network. Second, the accuracy before personalized training on the new test node ranges from 45% to 55%. This discrepancy is due to the suboptimal use of the global model as the initial model for the edge node, which is addressed through personalized training. As the demand for personalization decreases, the global model approaches the optimal solution, leading to an increase in test accuracy after personalized training, from 79.6% to 90.3%. Finally, the experiment results in Table I demonstrate that the CNN, LSTM, and CNN-BiLSTM

networks achieve local personalization improvements in edge devices. Although the LSTM network exhibits higher test accuracy than CNN (1.4%-2.9% improvement), the relative performance of the same network remains consistent.

C. The Performance of Global Model Test

After investigating PerFLID's personalization following several rounds of local learning, we conducted additional experiments to determine whether PerFLID's global model can handle complex global scenarios during the training phase. Using the same three datasets, we tested the accuracy of the global model and compared it with the latest meta-learning methods [46], the results are shown in Table II.

D. The Verification of Node Selection Strategies

This experiment is used to evaluate the effectiveness of the key component of PerFLID, i.e., to demonstrate that a node selection strategy based on cluster selection accelerates the convergence of the model. We trained using a dataset with 120 nodes, with a data distribution parameter of $\alpha = 0.1$ and other parameters identical to those in Section VI-A. To validate the generality of our node selection strategy, we compare three approaches to achieve intrusion detection: federated meta-learning [23], federated learning [45], and PerFLID. The

TABLE I
EXPERIMENTAL RESULTS AND COMPARATIVE EXPERIMENTAL RESULTS

Method	Dataset	$\alpha = 0.1$			$\alpha = 0.5$			$\alpha = 1$			$\alpha = 10$		
		Acc	Recall	F1-score	Acc	Recall	F1-score	Acc	Recall	F1-score	Acc	Recall	F1-score
DeepFed [16]	UNSW-NB15	0.689	0.557	0.583	0.723	0.653	0.615	0.776	0.705	0.725	0.903	0.889	0.896
	CIC-IDS2017	0.715	0.661	0.678	0.746	0.723	0.746	0.790	0.764	0.778	0.921	0.916	0.919
	BoT-IoT	0.657	0.521	0.596	0.696	0.610	0.635	0.740	0.774	0.787	0.873	0.841	0.839
FSLAD [21]	UNSW-NB15	0.695	0.612	0.637	0.741	0.681	0.708	0.785	0.739	0.754	0.905	0.892	0.901
	CIC-IDS2017	0.720	0.653	0.686	0.763	0.678	0.709	0.799	0.772	0.781	0.923	0.920	0.920
	BoT-IoT	0.663	0.583	0.606	0.713	0.620	0.649	0.750	0.701	0.723	0.875	0.845	0.847
Fed-ANIDS [44]	UNSW-NB15	0.772	0.687	0.705	0.795	0.708	0.741	0.811	0.789	0.801	0.908	0.892	0.905
	CIC-IDS2017	0.797	0.683	0.729	0.818	0.737	0.764	0.825	0.809	0.812	0.926	0.921	0.924
	BoT-IoT	0.739	0.643	0.702	0.768	0.698	0.717	0.775	0.726	0.738	0.879	0.863	0.857
SSFL [45]	UNSW-NB15	0.757	0.712	0.735	0.791	0.752	0.776	0.819	0.794	0.801	0.910	0.899	0.908
	CIC-IDS2017	0.782	0.738	0.759	0.814	0.769	0.784	0.833	0.801	0.820	0.928	0.921	0.925
	BoT-IoT	0.724	0.622	0.650	0.764	0.708	0.730	0.783	0.735	0.761	0.880	0.871	0.872
PerFLID	UNSW-NB15	0.817	0.776	0.792	0.825	0.806	0.818	0.836	0.802	0.810	0.913	0.905	0.910
	CIC-IDS2017	0.842	0.806	0.817	0.847	0.829	0.832	0.850	0.827	0.841	0.931	0.935	0.928
	BoT-IoT	0.785	0.759	0.773	0.798	0.774	0.783	0.801	0.749	0.762	0.884	0.868	0.871

TABLE II
COMPARISON ACCURACY RESULTS OF GLOBAL MODEL
ON THREE DATASETS

Methods	UNSW-NB15	CIC-IDS2017	BoT-IoT
DeepFed [16]	92.69%	93.89%	90.29%
FSLAD [21]	89.02%	90.47%	87.66%
Fed-ANIDS [44]	90.11%	91.53%	88.49%
SSFL [45]	95.60%	94.35%	90.72%
FC-Net [46]	92.64%	95.38%	93.86%
PerFLID	95.79%	96.53%	94.11%

experiments aim to evaluate the local adaptation and runtime of the trained models. The results of the experiments are shown in Table III, where the average running time of the algorithms intuitively shows that our approach runs faster while achieving similar detection accuracy. Thus, our model exhibits faster convergence compared to the other two methods, requiring only 4–10 epochs of personalized training to converge for each node with different data distributions. This observation aligns with our theoretical analysis, indicating efficient convergence without additional overhead. The last two columns of Table III present the experimental results of applying our node selection strategy to PFTD based on federated meta-learning and SSFL based on federated learning. It is evident that our node selection strategy significantly accelerates the training speed of each model, while also enhancing global accuracy and improving adaptation to local data.

E. The Additional Participant Scenario Test

Since different federated learning parameters affect the model training effect, for the presence of additional nodes involved in training, we compared the impact of different numbers of edge nodes on the model training accuracy when the node data heterogeneity parameter $\alpha = 0.5$. The experimental results are shown in the Figure 4. We also increased the learning rate and conducted tests after enough rounds of personalized training. Two conclusions can be drawn:

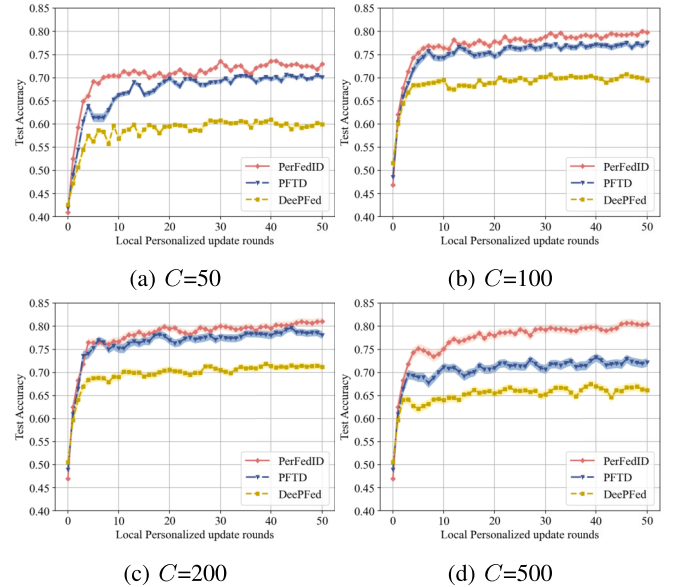


Fig. 4. The impact of additional participant nodes C in federated training test on UNSW-NB15 dataset.

1) *Impact of Additional Participant Nodes:* The average accuracy of the personalized model increases with the number of edge nodes. This is because with more edge nodes, the global model can access a greater number and variety of sample features. However, this improvement is not infinite and may even become counterproductive. When the total number of nodes reaches 500, the accuracy of our model plateaus, while the accuracy of PFTD drops by 5.42%. We believe that when the edge nodes cover a sufficient number of samples of all types, the model accuracy reaches its peak. Further increasing the number of edge nodes only increases storage overhead and model aggregation delay at the central server. Moreover, without a node selection strategy, increasing the number of edge nodes may result in certain types of samples not being selected for aggregation, thereby reducing model accuracy.

TABLE III
ACCURACY FOR THE EXPERIMENTS OF THE GLOBAL AND LOCAL SCENARIO

Methods	Accuracy of Global			Accuracy of Local Adaptation			Average times(s)
	UNSW-NB15	CIC-IDS2017	BoT-IoT	UNSW-NB15	CIC-IDS2017	BoT-IoT	
PerFLID	95.79%	96.53%	94.11%	89.54%	91.23%	90.59%	83.24
PFTD [23]	92.01%	93.68%	91.22%	85.57%	86.42%	82.31%	429.52
SSFL [45]	95.60%	94.35%	90.72%	74.13%	77.39%	78.25%	172.85
PFTD [23] with Node Select	92.76%	93.91%	91.45%	85.73%	86.75%	82.83%	252.46
SSFL [45] with Node Select	95.64%	94.38%	90.77%	75.28%	78.50%	78.94%	81.76

2) *Local Personalization*: For the globally learned model, conducting multiple epochs (more than 20) of local personalization is meaningless. Our proposed model quickly converges locally during personalized training on edge devices. Additional rounds of learning do not significantly improve model accuracy and may lead to overfitting, consuming local device resources unnecessarily.

VII. CONCLUSION AND FUTURE WORKS

A novel personalized federated learning framework, PerFLID, was proposed for detecting traffic intrusion in IoT nodes. Using meta-learning, the central model learns data characteristics from edge devices to create adaptive models, enabling personalized intrusion detection against unknown traffic attacks. To accelerate deployment, we designed a node selection strategy based on local gradient feature clustering, theoretically proving it speeds up convergence and improves efficiency. We demonstrated the framework's convergence upper bound and conducted extensive simulations with various neural network models on three NIDS datasets. Results show the personalized model has local adaptability and faster training convergence. Experiments confirm that our personalized federated learning intrusion detection framework achieves state-of-the-art performance in federated machine learning scenarios, meets the personalized needs of edge devices, and maintains optimal local attack detection in distributed environments.

Currently, PerFLID primarily addresses conventional IoT threats, but detecting advanced persistent threats (APTs) in IoT environments remains a challenge due to their time-sensitive and covert nature. Future research will focus on enhancing our federated learning framework to address APT characteristics by developing new detection methods, incorporating time series analysis techniques, and improving real-time detection capabilities to ensure timely responses.

REFERENCES

- [1] B. L. R. Stojkoska and K. V. Trivodaliev, "A review of Internet of Things for smart home: Challenges and solutions," *J. Cleaner Prod.*, vol. 140, pp. 1454–1464, Jan. 2017.
- [2] H. Boyes, B. Hallaq, J. Cunningham, and T. Watson, "The industrial Internet of Things (IIoT): An analysis framework," *Comput. Ind.*, vol. 101, pp. 1–12, Oct. 2018.
- [3] M. M. Islam, A. Rahaman, and M. R. Islam, "Development of smart healthcare monitoring system in IoT environment," *Social Netw. Comput. Sci.*, vol. 1, no. 3, pp. 1–11, May 2020.
- [4] M. Humayun, N. Jhanjhi, B. Hamid, and G. Ahmed, "Emerging smart logistics and transportation using IoT and blockchain," *IEEE Internet Things Mag.*, vol. 3, no. 2, pp. 58–62, Jun. 2020.
- [5] M. Antonakakis et al., "Understanding the Mirai botnet," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 1093–1110.
- [6] B. Seri and A. Livne. (2019). *Exploiting Blueborne in Linux-Based IoT Devices*. [Online]. Available: <https://media.armis.com/pdfs/wp-exploiting-blueborne-in-linuxbased-iot-devices-en.pdf>
- [7] N. Moustafa, B. Turnbull, and K.-K. R. Choo, "An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4815–4830, Jun. 2018.
- [8] M. F. Umer, M. Sher, and Y. Bi, "Flow-based intrusion detection: Techniques and challenges," *Comput. Secur.*, vol. 70, pp. 238–254, Sep. 2017.
- [9] E. Aydin and S. Bahtiyar, "OCIDS: An online CNN-based network intrusion detection system for DDoS attacks with IoT botnets," in *Proc. 14th Int. Conf. Secur. Inf. Netw. (SIN)*, Dec. 2021, pp. 1–8.
- [10] S. Seth, G. Singh, and K. Kaur Chahal, "A novel time efficient learning-based approach for smart intrusion detection system," *J. Big Data*, vol. 8, no. 1, p. 111, Dec. 2021.
- [11] C. Zhang, D. Jia, L. Wang, W. Wang, F. Liu, and A. Yang, "Comparative research on network intrusion detection methods based on machine learning," *Comput. Secur.*, vol. 121, Oct. 2022, Art. no. 102861.
- [12] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava, "Federated-learning-based anomaly detection for IoT security attacks," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2545–2554, Feb. 2021.
- [13] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," *Proc. Artif. Intell. Statist.*, vol. 3, pp. 1273–1282, May 2017.
- [14] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, "Network intrusion detection for IoT security based on learning techniques," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2671–2701, 3rd Quart., 2019.
- [15] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DIIoT: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, May 2019, pp. 756–767.
- [16] B. Li, Y. Wu, J. Song, R. Lu, T. Li, and L. Zhao, "DeepFed: Federated deep learning for intrusion detection in industrial Cyber-Physical systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5615–5624, Aug. 2021.
- [17] S. Chatterjee and M. K. Hanawal, "Federated learning for intrusion detection in IoT security: A hybrid ensemble approach," *Int. J. Internet Things Cyber-Assurance*, vol. 1, no. 1, p. 1, 2023.
- [18] H. Liu et al., "Blockchain and federated learning for collaborative intrusion detection in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 6073–6084, Jun. 2021.
- [19] A. Z. Tan, H. Yu, L. Cui, and Q. Yang, "Towards personalized federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 3, no. 1, pp. 1–17, Apr. 2022.
- [20] V. Kulkarni, M. Kulkarni, and A. Pant, "Survey of personalization techniques for federated learning," in *Proc. 4th World Conf. Smart Trends Syst., Secur. Sustain. (WorldS4)*, 2020, pp. 794–797.
- [21] O. Aouedi, K. Piamrat, G. Muller, and K. Singh, "Federated semisupervised learning for attack detection in industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 286–295, Jan. 2023.
- [22] R. Zhao, Y. Yin, Y. Shi, and Z. Xue, "Intelligent intrusion detection based on federated learning aided long short-term memory," *Phys. Commun.*, vol. 42, Oct. 2020, Art. no. 101157.

- [23] Y. Hu, J. Wu, G. Li, J. Li, and J. Cheng, "Privacy-preserving few-shot traffic detection against advanced persistent threats via federated meta learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2549–2560, May 2024.
- [24] H. Ding, L. Chen, S. Li, Y. Bai, P. Zhou, and Z. Qu, "Divide, conquer, and coalesce: Meta parallel graph neural network for IoT intrusion detection at scale," in *Proc. ACM Web Conf.*, May 2024, pp. 1656–1667.
- [25] A. Back de Luca, G. Zhang, X. Chen, and Y. Yu, "Mitigating data heterogeneity in federated learning with data augmentation," 2022, *arXiv:2206.09979*.
- [26] Y. Fraboni, R. Vidal, L. Kameni, and M. Lorenzi, "Clustered sampling: Low-variance and improved representativity for clients selection in federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 3407–3416.
- [27] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. 34th Int. Conf. Mach. Learn.*, vol. 70, Aug. 2017, pp. 1126–1135.
- [28] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning: A meta-learning approach," 2020, *arXiv:2002.07948*.
- [29] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-IID data silos: An experimental study," in *Proc. IEEE 38th Int. Conf. Data Eng. (ICDE)*, May 2022, pp. 965–978.
- [30] S. Moon and W. H. Lee, "Privacy-preserving federated learning in Healthcare," in *Proc. Int. Conf. Electron., Inf., Commun. (ICEIC)*, 2023, pp. 1–4.
- [31] T.-M. Harry Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," 2019, *arXiv:1909.06335*.
- [32] X. Gong, Y. Chen, Q. Wang, and W. Kong, "Backdoor attacks and defenses in federated learning: State-of-the-art, taxonomy, and future directions," *IEEE Wireless Commun.*, vol. 30, no. 2, pp. 114–121, Apr. 2023.
- [33] S. Huang, Y. Li, C. Chen, L. Shi, and Y. Gao, "Multi-metrics adaptively identifies backdoors in federated learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 4629–4639.
- [34] Y. Wang, D. Zhai, and Y. Xia, "SCFL: Mitigating backdoor attacks in federated learning based on SVD and clustering," *Comput. & Secur.*, vol. 133, Aug. 2023, Art. no. 103414.
- [35] J. Wolfrath, N. Sreekumar, D. Kumar, Y. Wang, and A. Chandra, "HACCS: Heterogeneity-aware clustered client selection for accelerated federated learning," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, May 2022, pp. 985–995.
- [36] K. Muhammad et al., "FedFast: Going beyond average for faster training of federated recommender systems," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2020, pp. 1234–1242.
- [37] H. T. Nguyen, V. Sehwag, S. Hosseinalipour, C. G. Brinton, M. Chiang, and H. Vincent Poor, "Fast-convergent federated learning," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 201–218, Jan. 2021.
- [38] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proc. Mach. Learn. Syst.*, vol. 2, pp. 429–450, May 2020.
- [39] Y. Arjevani, Y. Carmon, J. C. Duchi, D. J. Foster, N. Srebro, and B. Woodworth, "Lower bounds for non-convex stochastic optimization," *Math. Program.*, vol. 199, nos. 1–2, pp. 165–214, May 2023.
- [40] J. Wang, Q. Liu, H. Liang, G. Joshi, and V. H. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Proc. NIPS*, 2020, pp. 7611–7623.
- [41] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, Nov. 2015, pp. 1–6.
- [42] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISp*, vol. 1, May 2018, pp. 108–116.
- [43] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, Nov. 2019.
- [44] M. J. Idrissi et al., "Fed-ANIDS: Federated learning for anomaly-based network intrusion detection systems," *Expert Syst. Appl.*, vol. 234, Dec. 2023, Art. no. 121000.
- [45] R. Zhao, Y. Wang, Z. Xue, T. Ohtsuki, B. Adebisi, and G. Gui, "Semisupervised federated-learning-based intrusion detection method for Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8645–8657, May 2023.
- [46] C. Xu, J. Shen, and X. Du, "A method of few-shot network intrusion detection based on meta-learning framework," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3540–3552, 2020.